ESP32-CAM Surveillance Streaming Device Requirements

1. Project Objective

Develop a surveillance video streaming device using an AI Thinker ESP32-CAM programmed through the Arduino IDE.

The device should function as a standalone IP camera that connects to a Wi-Fi network, serves live video through a web interface, supports multiple simultaneous viewers, and provides basic camera controls without requiring the firmware to be reflashed.

---

2. Network Requirements

2.1 Static Network Configuration

On boot, the ESP32-CAM shall configure the following static network settings:

Setting	Value
IP Address	192.168.1.99
Gateway	192.168.1.1
Subnet Mask	255.255.255.0
DNS Server	8.8.8.8

2.2 Wi-Fi Connection

- Automatically connect to the configured Wi-Fi network on startup.
- Retry connection if Wi-Fi is unavailable.
- Display connection status on the serial monitor.
- Print the assigned IP address after successful connection.

---

3. Video Streaming Requirements

3.1 Stream Resolution

The camera shall stream at:

- Resolution: **SVGA (800 × 600)**

If the camera cannot reliably sustain this resolution, the firmware should automatically fall back to the next supported resolution while logging the reason.

3.2 Video Quality

- Continuous MJPEG streaming.
- Smooth frame updates.
- Maintain stable streaming without excessive frame drops.
- Optimize JPEG compression for image clarity while balancing bandwidth.

3.3 Multiple Client Support

Unlike the default ESP32 CameraWebServer example, the application shall:

- Allow multiple browsers to connect simultaneously.
- Each client shall receive an independent MJPEG stream.
- One client disconnecting shall not interrupt streams for other clients.
- The maximum number of concurrent viewers should be configurable.

3.4 Browser Compatibility

The web interface shall function correctly on:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Mobile browsers (optional but preferred)

---

4. Web Interface

The ESP32-CAM shall host its own web page.

Layout

The web page should contain:

```
--------------------------------------------------
             ESP32 Surveillance Camera
```

```
            [ Live Camera Stream ]

-----------------------------------------------------

Brightness     [--------O------]

Contrast       [-----O---------]

Saturation     [-------O-------]

Restart Camera
-----------------------------------------------------
```

The live stream should remain centered regardless of browser size.

---

5. Camera Controls

The web page shall expose the following controls.

Brightness Slider

Adjust brightness in real time.

Expected range:

- -2
- -1
- 0
- +1
- +2

Changes should apply immediately without restarting the stream.

Contrast Slider

Adjust image contrast.

Expected range:

- -2
- -1
- 0
- +1
- +2

Saturation Slider

Adjust image saturation.

Expected range:

- -2
- -1
- 0
- +1
- +2

Restart Camera Button

Provide a button that:

- Stops the current camera stream.
- Reinitializes the camera driver.
- Restarts the MJPEG stream.
- Reloads the video automatically after restart.
- Does not require rebooting the ESP32.

If camera restart fails, display an appropriate error.

---

6. Performance Requirements

The firmware should:

- Maintain stable streaming for long durations.
- Recover from temporary streaming failures.
- Prevent browser freezes.
- Handle temporary network interruptions.
- Recover automatically after client disconnects.

Memory leaks should be avoided.

---

7. Camera Initialization

During startup:

- Detect camera successfully.
- Initialize PSRAM if available.

- Configure JPEG quality.
- Configure frame size.
- Verify camera initialization before starting the web server.

If initialization fails:

- Print the error to Serial.
- Retry initialization (optional).
- Avoid crashing.

---

8. Web Server Requirements

The embedded web server shall expose endpoints for:

Endpoint	Purpose
/	Main webpage
/stream	MJPEG video stream
/brightness	Update brightness
/contrast	Update contrast
/saturation	Update saturation
/restart	Restart camera

Additional endpoints may be added later.

---

9. User Experience

The web interface should:

- Load quickly.
- Automatically begin streaming after page load.
- Not require manual refresh after changing settings.
- Be responsive on different screen sizes.
- Have a clean, minimal interface.

---

10. Reliability

The firmware should recover gracefully from:

- Wi-Fi reconnection

- Browser refresh
- Client disconnect
- Camera restart
- Temporary camera capture failures

No firmware reboot should be required for normal recovery.

---

11. Logging

Useful debug messages shall be printed to the serial monitor, including:

- Boot completed
- Connecting to Wi-Fi
- Wi-Fi connected
- Camera initialized
- Camera initialization failed
- Stream client connected
- Stream client disconnected
- Camera restarted
- Camera restart failed
- Memory information (optional)

---

12. Future Expandability

The firmware architecture should make it straightforward to add features such as:

- Motion detection
- Snapshot capture
- Video recording
- Authentication
- HTTPS
- OTA firmware updates
- Pan/Tilt support (external servos)
- Multiple camera synchronization
- REST API integration

---

13. Non-Functional Requirements

- Written entirely in Arduino IDE compatible C++.
- Compatible with the AI Thinker ESP32-CAM module.
- Use only actively maintained Arduino-compatible libraries.
- Minimize RAM usage.
- Minimize CPU utilization.

- Stable operation for continuous 24/7 surveillance use.
- Source code should be modular and well documented for future enhancements.

Additional Requirements I Added

Compared to your original list, I added the following items because they are important for a robust surveillance solution:

1. **Subnet mask specification** (required for static IP configuration).
2. **Automatic Wi-Fi reconnection** if the network is temporarily unavailable.
3. **Serial logging** for debugging and monitoring.
4. **Fallback behavior** if 800×600 cannot be sustained due to hardware limitations.
5. **Independent support for multiple simultaneous viewers**, so one client cannot interrupt another.
6. **Automatic stream recovery** after browser refreshes or temporary network issues.
7. **Explicit camera initialization and failure handling** to avoid crashes.
8. **Defined HTTP endpoints** for stream and camera controls, making the firmware easier to extend.
9. **Long-running stability requirements**, including avoidance of memory leaks and 24/7 operation.
10. **Responsive web interface** that automatically starts streaming and remains centered.
11. **Modular, maintainable code structure** to support future enhancements.
12. **Future extensibility considerations**, such as OTA updates, authentication, motion detection, and recording.

One additional recommendation is to use the asynchronous networking stack (e.g., AsyncTCP + ESPAsyncWebServer or an equivalent maintained async server library) rather than the basic synchronous web server. Supporting multiple MJPEG clients on an ESP32-CAM is significantly more reliable with an asynchronous architecture, especially if you plan to have several viewers connected simultaneously.